UNIVERSIDAD POLITÉCNICA BICENTENARIO

Práctica 4: Regulador de brillo mediante ESP32, servomotor y dímer con TRIAC

Presenta:

Jesús Ángel Bonilla Duran

Viridiana Santibáñez Carrillo

Jocelyn Rodriguez Torres

Matricula:

24033407

24033066

24033436

Asignatura:

Sistemas Embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao 15 de octubre de 2026

Contenido

Introducción 3
Marco Teórico 3
Simulación 3
Sistemas de control automatizado 3
Microcontrolador ESP32 4
Entradas digitales y botones 4
Servomotor 4
Dímer con TRIAC 4
Pantalla LCD con comunicación I2C 5
Cálculo del porcentaje de brillo 5
Desarrollo 5
Material requerido 5
Configuración de pines 6
Descripción del código 7
Funcionamiento general del programa 8
Resultados 8
Observaciones y Conclusiones 10
Referencias 11

Lista de Figuras

Figura 1 Diagrama de conexión de los componentes del sistema de control en Wokwi. 6
Figura 2 Montaje físico del servomotor colocado en la perilla del dímer con TRIAC. 9
Figura 3 Pantalla LCD mostrando el porcentaje de brillo calculado y el ángulo del servomotor. 9
Figura 4 Prueba de funcionamiento del sistema de regulación de brillo. 10
Figura 5 10

Introducción

En esta práctica se desarrolló un sistema de control para regular el brillo de una lámpara mediante un ESP32, dos botones, un servomotor y una pantalla LCD de 16×2 con comunicación I2C. El sistema permite aumentar o disminuir el nivel de brillo utilizando los botones, mientras que el servomotor gira la perilla de un dímer con TRIAC para modificar físicamente la intensidad luminosa.

Además, se utiliza una pantalla LCD para mostrar el porcentaje de brillo indicado por el programa y el ángulo de posición del servomotor. Con esta práctica se aplican conocimientos de programación, entradas digitales, control de actuadores y visualización de información, integrando los componentes para lograr un sistema de regulación automatizado.

Marco Teórico

Simulación

Wokwi es una plataforma en línea que permite diseñar, programar y simular proyectos electrónicos directamente desde un navegador web la herramienta permite trabajar con diferentes placas de desarrollo, entre ellas Arduino, ESP32, STM32 y Raspberry Pi Pico, además de integrar componentes electrónicos y conexiones para comprobar virtualmente el funcionamiento de un circuito (Group, 2023).

En este proyecto se utilizó Wokwi para diseñar y simular los circuitos empleados con el ESP32, permitiendo realizar pruebas del programa y verificar las conexiones de los componentes antes de llevar a cabo el montaje físico de esta manera, la simulación ayudó a detectar posibles errores de programación o conexión durante el desarrollo del proyecto.

Sistemas de control automatizado

Un sistema de control automatizado permite modificar el funcionamiento de un dispositivo mediante instrucciones programadas, con poca intervención directa del usuario. Para lograrlo, se utilizan elementos de entrada, un controlador y actuadores que ejecutan las acciones indicadas.

En esta práctica, los botones funcionan como entradas, el ESP32 procesa las instrucciones y el servomotor actúa sobre la perilla del dímer para regular el brillo de la lámpara.

Microcontrolador ESP32

El ESP32 es un microcontrolador que cuenta con entradas y salidas digitales, además de diferentes interfaces de comunicación. Puede programarse para recibir señales de botones, procesar información y controlar dispositivos electrónicos.

En el sistema desarrollado, el ESP32 interpreta las pulsaciones de los botones, calcula la posición del servomotor y actualiza la información mostrada en la pantalla LCD.

Entradas digitales y botones

Los botones permiten que el usuario envíe instrucciones al microcontrolador. En el programa se utiliza la configuración INPUT_PULLUP, que activa una resistencia interna de polarización.

Con esta configuración, cuando el botón no está presionado, la entrada normalmente se encuentra en estado HIGH; cuando se presiona, cambia a LOW. Esto permite identificar cuándo se solicita aumentar o disminuir el brillo.

Servomotor

Un servomotor es un actuador que permite controlar la posición angular de su eje. En esta práctica se utiliza para girar la perilla del dímer, modificando su posición para aumentar o reducir el brillo de la lámpara.

El programa establece un ángulo entre 0° y 180°. Cada pulsación modifica la posición en intervalos de 2°, siempre que no se haya alcanzado el límite establecido.

Dímer con TRIAC

Un dímer con TRIAC es un dispositivo electrónico utilizado para regular la potencia suministrada a una carga de corriente alterna, como una lámpara compatible. Su funcionamiento se basa comúnmente en controlar el momento de conducción del TRIAC durante cada semiciclo de la señal de corriente alterna.

En esta práctica, el servomotor gira físicamente la perilla del dímer. De esta manera, el sistema modifica el ajuste del dispositivo y permite regular la intensidad luminosa de la lámpara. El TRIAC realiza la regulación eléctrica, mientras que el servomotor permite automatizar el movimiento de la perilla.

Pantalla LCD con comunicación I2C

La pantalla LCD de 16×2 permite mostrar información mediante dos filas de 16 caracteres. La comunicación I2C reduce la cantidad de conexiones necesarias al utilizar principalmente las líneas SDA y SCL, además de alimentación y tierra.

En el proyecto, la pantalla muestra el mensaje de inicio “REGULADOR DIMER” y, en la segunda fila, el porcentaje de brillo calculado y el ángulo del servomotor.

Cálculo del porcentaje de brillo

El programa relaciona el ángulo del servomotor con un porcentaje de referencia mediante la función map(). La conversión utilizada es:

$$\boldsymbol{Porcentaje = \frac{Angulo \times 100}{180}}$$

Por ejemplo, para un ángulo de 90°:

$$\boldsymbol{Porcentaje = \frac{90 \times 100}{180}}$$

Este porcentaje representa el valor calculado por el programa a partir de la posición del servomotor. No corresponde a una medición directa de la intensidad luminosa, ya que el brillo real también depende del dímer y de la lámpara utilizada.

Desarrollo

Material requerido

- 1 tarjeta ESP32.
- 1 Foco
- 1 servomotor
- 2 botones pulsadores
- 1 pantalla LCD de 16×2 con I2C
- 1 Dimer con TRIAC
- 1 Socket
- Cables de conexión
- 1 fuente de alimentación

- Protoboard
- Computadora con Visual Studio Code y PlatformIO.
- Simulador Wokwi.

Configuración de pines

Tabla 1. Configuracion de pines

Componente	Pin del ESP32	Configuración o función
Servomotor	GPIO 18	Señal de control.
Botón para aumentar	GPIO 12	Entrada digital con INPUT_PULLUP.
Botón para disminuir	GPIO 14	Entrada digital con INPUT_PULLUP.
LCD – SDA	GPIO 21	Línea de datos I2C, si se utilizó la configuración predeterminada.
LCD – SCL	GPIO 22	Línea de reloj I2C, si se utilizó la configuración predeterminada.

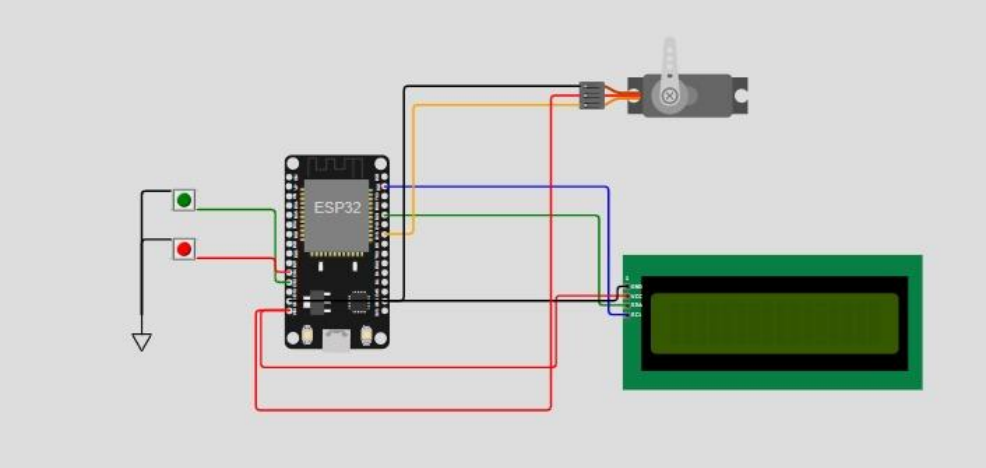


Figura 1 Diagrama de conexión de los componentes del sistema de control en Wokwi.

A continuación, se muestra el código empleado para el desarrollo de la práctica.

main.cpp

```
#include <Arduino.h>
#include <ESP32Servo.h>
#include <LiquidCrystal_I2C.h>

Servo servo;
// Dirección I2C común: 0x27 (o 0x3F), 16 columnas x 2 filas
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define SERVO_PIN 18
#define BTN_AUMENTAR 12  // Girar a la derecha (Aumentar brillo)
#define BTN_DISMINUIR 14 // Girar a la izquierda (Reducir brillo)

int angulo = 0;          // Posición inicial del servo (0 a 180)
int anguloAnterior = -1;  // Detección de cambios

void setup() {
    Serial.begin(115200);

    // Botones con resistencia PULLUP interna (no requieren resistencia externa)
    pinMode(BTN_AUMENTAR, INPUT_PULLUP);
    pinMode(BTN_DISMINUIR, INPUT_PULLUP);

    // Configuración del servo
    servo.attach(SERVO_PIN);
    servo.write(angulo);
```

```
    // Configuración de la pantalla LCD
    lcd.init();
    lcd.backlight();

    // Mensaje de bienvenida fija en la primera fila
    lcd.setCursor(0, 0);
    lcd.print(" REGULADOR DIMER ");
}

void loop() {
    bool huboCambio = false;

    // --- Botón Aumentar / Derecha ---
    if (digitalRead(BTN_AUMENTAR) == LOW) {
        if (angulo < 180) {
            angulo += 2; // Incrementar de 2 en 2 grados para un movimiento fluido
            if (angulo > 180) angulo = 180;
            huboCambio = true;
        }
        delay(35); // Controla la velocidad del giro al mantener presionado
    }

    // --- Botón Disminuir / Izquierda ---
    if (digitalRead(BTN_DISMINUIR) == LOW) {
        if (angulo > 0) {
            angulo -= 2; // Decrementar de 2 en 2 grados
            if (angulo < 0) angulo = 0;
            huboCambio = true;
        }
        delay(35); // Controla la velocidad del giro al mantener presionado
    }

    if (huboCambio || angulo != anguloAnterior) {
        servo.write(angulo);
        int porcentaje = map(angulo, 0, 180, 0, 100);

        lcd.setCursor(0, 1);
        lcd.print("Brillo:");
        if (porcentaje < 100) lcd.print(" ");
        if (porcentaje < 10)  lcd.print(" ");
        lcd.print(porcentaje);
        lcd.print("% ");

        if (angulo < 100) lcd.print(" ");
        if (angulo < 10)  lcd.print(" ");
        lcd.print(angulo);
        lcd.print((char)223); // Muestra el símbolo °

        anguloAnterior = angulo;
    }
}
```

Descripción del código

Instrucción o elemento	Descripción
`#include <Arduino.h>`	Permite utilizar las funciones básicas de Arduino.
`#include <ESP32Servo.h>`	Permite controlar el servomotor con el ESP32.
`#include <LiquidCrystal_I2C.h>`	Permite controlar la pantalla LCD mediante I2C.
`Servo servo;`	Crea el objeto para controlar el servomotor.
`LiquidCrystal_I2C lcd(0x27, 16, 2);`	Configura la pantalla LCD con dirección I2C 0x27 y tamaño de 16×2.
`#define SERVO_PIN 18`	Define el pin de control del servomotor.
`#define BTN_AUMENTAR 12`	Define el botón para aumentar el brillo.
`#define BTN_DISMINUIR 14`	Define el botón para disminuir el brillo.
`pinMode(..., INPUT_PULLUP)`	Configura los botones con resistencias internas de polarización.
`servo.attach(SERVO_PIN)`	Asocia el servomotor al pin indicado.
`lcd.init()` y `lcd.backlight()`	Inicializan la pantalla y encienden su iluminación.
`digitalRead()`	Lee el estado de los botones.
`angulo += 2`	Aumenta el ángulo del servomotor en 2°.

`angulo -= 2`	Disminuye el ángulo del servomotor en 2°.
`servo.write(angulo)`	Ordena al servomotor colocarse en el ángulo indicado.
`map(angulo, 0, 180, 0, 100)`	Convierte el ángulo en un porcentaje de referencia.
`lcd.print()`	Muestra mensajes, porcentaje y ángulo en la pantalla.

Funcionamiento general del programa

El programa inicia la comunicación serial a 115200 baudios y configura los botones como entradas digitales con resistencias internas INPUT_PULLUP. Después, habilita el servomotor conectado al GPIO 18 e inicializa la pantalla LCD.

En el ciclo principal, el programa revisa continuamente el estado de los botones. Si se presiona el botón de aumentar, el ángulo se incrementa en 2°; si se presiona el botón de disminuir, se reduce en 2°. El valor se mantiene dentro del rango de 0° a 180°.

Cuando ocurre un cambio, el servomotor se mueve a la nueva posición y el programa calcula el porcentaje correspondiente. Finalmente, la pantalla muestra el porcentaje y el ángulo actualizado.

Resultados

Se desarrolló un sistema de control que permite modificar la posición de un servomotor mediante dos botones conectados al ESP32. El programa establece límites de movimiento de 0° a 180° y actualiza la pantalla LCD con el porcentaje calculado y el ángulo correspondiente.

En el montaje físico, el servomotor se utiliza para girar la perilla del dímer con TRIAC, permitiendo modificar el ajuste de brillo de la lámpara. De esta forma, se integra el control digital del ESP32 con un actuador mecánico y un dispositivo de regulación de potencia.

La simulación en Wokwi permite representar y verificar la programación de los componentes disponibles. Sin embargo, debido a que no se dispone del dímer con TRIAC utilizado en el montaje físico, la regulación real de la lámpara no se representa directamente en la simulación.

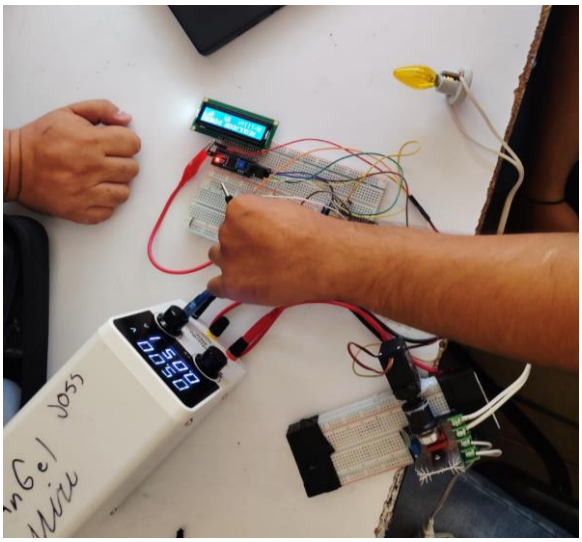


Figura 2 Montaje físico del servomotor colocado en la perilla del dímer con TRIAC.

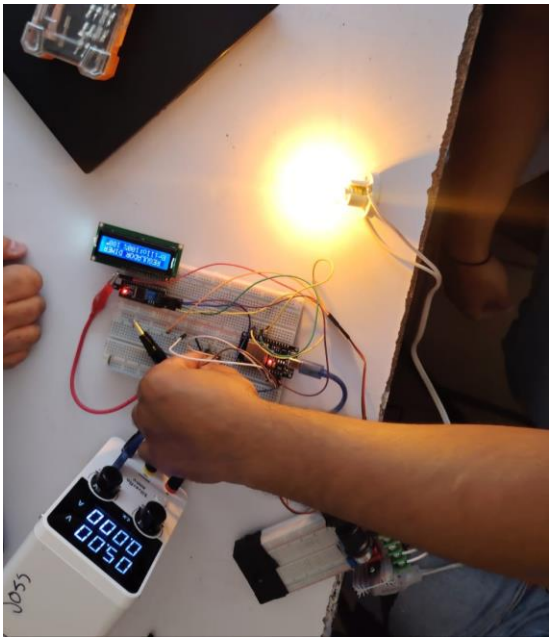


Figura 3 Pantalla LCD mostrando el porcentaje de brillo calculado y el ángulo del servomotor.

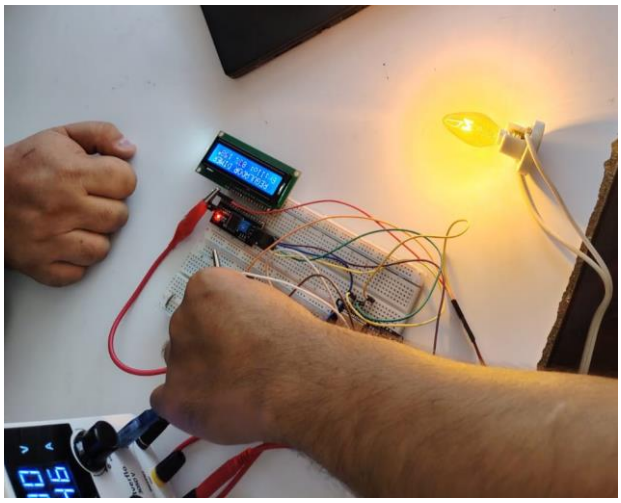


Figura 4 Prueba de funcionamiento del sistema de regulación de brillo.

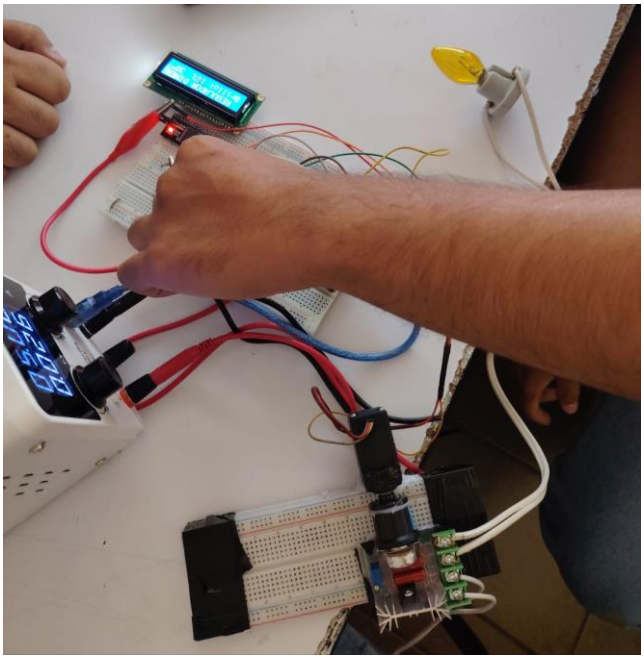


Figura 5

Observaciones y Conclusiones

Durante la práctica se observó que el ESP32 permite controlar la posición del servomotor mediante dos botones pulsadores, el botón de aumentar incrementa el ángulo en 2°, mientras que el botón de disminuir lo reduce en la misma cantidad, manteniéndose dentro del rango de 0° a 180° también se utilizó una pantalla LCD de 16×2 para mostrar el porcentaje calculado y el ángulo del servomotor, este movimiento permite girar la perilla del dímer con TRIAC para ajustar el brillo de la lámpara. Sin embargo, Wokwi no cuenta con el dímer utilizado en el montaje físico, por lo que no fue posible representarlo directamente en

la simulación. Además, el porcentaje mostrado corresponde a un cálculo basado en el ángulo y no a una medición real de la intensidad luminosa.

En esta práctica aprendimos a integrar diferentes componentes electrónicos para desarrollar un sistema de control mediante el ESP32. Comprendimos cómo utilizar botones pulsadores para modificar la posición de un servomotor y cómo mostrar información en una pantalla LCD, asimismo, relacionamos el movimiento mecánico del servomotor con el ajuste de la perilla del dímer con TRIAC para regular el brillo de una lámpara, aunque la simulación en Wokwi presentó limitaciones al no incluir el dímer utilizado físicamente, pudimos comprender la función de cada componente y la importancia de su correcta conexión y programación.

Referencias

[1] (Nick, s.f.) *The ESP32 chip explained: Advantages and applications*.

[2] PlatformIO. (s.f.). PlatformIO: Professional collaborative platform for embedded development.

[3] Maxi. (s.f.). Introducción a PlatformIO con Visual Studio Code.

[4] Group, u. (2023). Wokwi: Simulador en línea de Arduino, ESP32, STM32 y Raspberry Pi Pico.